

Operating Systems Laboratory (CSE 4510)

Week 3: Bash Shell Scripting

1. What is a Shell Script?

A shell script is a text file containing Linux commands that run automatically.

Concept	Meaning
Shell	Program that executes commands (Bash, Zsh, etc.)
Script	File containing commands
Extension	.sh
Interpreter	#!/bin/bash

2. Creating & Running a Script

Step	Command	Explanation
1	nano hello.sh or touch hello.sh	Create file
2	#!/bin/bash echo "Hello UIU Students!"	Add inside the file
3	chmod +x hello.sh	Make script executable
4	./hello.sh or bash file_name.ext	Run script

3. Variables in Bash

3.1 String Variable Initialization

Type	Example	Output
Set string	name="UIU Student"	–
Print string	echo \$name	UIU Student
String + variable	echo "Welcome, \$name"	Welcome, UIU Student
Concatenation	full="\$name Dept"	–

3.2 Integer Variable Initialization

Type	Example
Integer	x=10
Print	echo \$x
With text	echo "Value: \$x"

4. Mathematical Expressions (All Methods)

4.1 Integer Value Operations

Method	Example	Output
Double parentheses	echo \$((5+3))	8
expr command (int only)	expr 5 + 3	8
let command (int only)	let x=5+3; echo \$x	8
Using variables (int only)	a=5; b=3; echo \$((a+b))	8
Increment / Decrement	((x++)), ((x--))	–

Example: *expr* vs. *let*

expr	<pre>#!/bin/bash x=5 y=10 sum=\$(expr \$x + \$y) # use spaces around the + operator echo "\$sum"</pre>
let	<pre>#!/bin/bash a=20 b=4 let "add = a + b" let "sub = a - b" let "mul = a * b" let "div = a / b" echo "Addition = \$add" echo "Subtraction = \$sub" echo "Multiplication = \$mul" echo "Division = \$div"</pre>

4.2 Float Value Operations

For float values, "\$(())" cannot be used — instead use \$(echo "expr" | bc).

<pre>echo "10.5 + 2.5" bc</pre>	<pre>x=5.5 y=1.5 sum=\$(echo "\$x + \$y" bc) echo \$sum</pre>
-----------------------------------	---

5. Echo Commands (All Forms)

Purpose	Example	Output
Print text	echo "Hello"	Hello
Print variable	echo \$name	Value of name
Text + variable	echo "Hello \$name"	Hello UIU
Variable inside text	echo "Name: \${name}"	Name: UIU
Escape characters	echo -e "A\nB"	A (newline) B
No newline	echo -n "Loading..."	Loading...

6. Command Line Arguments

Bash	Run	Output
<pre>#!/bin/bash echo "Hello \$1 \$2"</pre>	bash file.sh Enamul Haque	Hello Enamul Haque

Special Argument Variables

Variable	Meaning	Notes
\$0	Script name	
\$1, \$2	First, second argument	Additional arguments: \$3, \$4, ..
\$@	All arguments passed	
\$#	Total number of arguments	
\$?	Exit status of last command	Returns 0 (success) or non-zero (1, 2, 127, ...)

Example

```
#!/bin/bash
echo "Script Name = $0"
echo "First Argument = $1"
echo "Second Argument = $2"
echo "All Arguments = @"
echo "Total Number of Arguments = $#"
```

```
ls
echo "Exit Status of ls = $?"
ls abc.txt
echo "Exit Status of previous command = $?"
```

Run using this:
bash test.sh Hi Hello

7. Special Variable: \$? (Exit Status)

Code 0 means success; a non-zero code (1, 2, 127, ...) means an error, with the specific value indicating the condition.

Example

```
ls file.txt
echo $?
```

If `ls file.txt` executes correctly, `echo` will print 0; otherwise, it prints a non-zero number.

8. Comparison Operators

8.1 Numeric Operators

Operator	Meaning
-lt	Less than
-gt	Greater than
-le	Less than or equal
-ge	Greater than or equal
-eq	Equal
-ne	Not equal

8.2 Logical Operators

Operator	Meaning
-a	AND
-o	OR
Example	if [\$x -gt 10 -a \$y -lt 5]

8.3 Writing Logical Expressions

Using square brackets with \$	x=12 y=2 if [\$x -gt 10 -a \$y -lt 5] then echo "\$x is greater than 10, \$y is less than 5" fi
Using double parentheses with \$	x=12 y=2 if ((\$x > 10 && \$y < 5)) then echo "\$x is greater than 10, \$y is less than 5" fi

Using double parentheses without \$	<pre>x=10 y=2 if ((x > 10 && y < 5)) then echo "\$x is greater than 10, \$y is less than 5" fi</pre>
-------------------------------------	--

8.4 User Input from Prompt

Way	Prompt Syntax	Full Example
1	<code>read -p "Enter number: " n</code>	<pre>#!/bin/bash read -p "Enter your name: " name echo "Hello, \$name"</pre>
2	<code>echo -n "Enter number: "</code> <code>read n</code>	<pre>#!/bin/bash echo -n "Enter your age: " read age echo "Your age is \$age"</pre>

9. File Test Operators

Flag	Meaning	Example	True When
-f	Regular file	<code>[-f sample.txt]</code>	File exists
-d	Directory	<code>[-d myfolder]</code>	Directory exists
-r	Readable	<code>[-r file]</code>	Read permission exists
-w	Writable	<code>[-w file]</code>	Write allowed
-x	Executable	<code>[-x script.sh]</code>	Execute allowed

10. If-Else Statements with Math Expressions

Syntax	<pre>if [condition] then commands else commands fi</pre>
Example	<pre>read -p "Enter number: " n if [\$n -lt 10] then echo "Small" elif [\$n -le 50] then echo "Medium" else echo "Large" fi</pre>

11. Examples

1) Even / Odd	<pre>read -p "Enter number: " n if ((n % 2 == 0)) then echo "Even" else echo "Odd" fi</pre>
----------------------	---

2) File Checker (-f, -d, -x, -r)	<pre>read -p "Enter path: " p if [-f \$p] then echo "It is a file" elif [-d \$p] then echo "It is a directory" else echo "Not found" fi</pre>
---	---

Practice Problems

1) Grade Checker

Input marks:

- $\geq 80 \rightarrow A$
- $\geq 60 \rightarrow B$
- $\geq 40 \rightarrow C$
- Else $\rightarrow$ Fail

2) Age Category

Input age:

- $< 13 \rightarrow$ Child
- $< 20 \rightarrow$ Teenager
- $< 60 \rightarrow$ Adult
- Else $\rightarrow$ Senior